Algorithms

Lecture #32

Syed Hamed Raza

Design & Analysis of Algorithms

DFS

Cycles

DFS-Tree Structures

DFS - Tree Structure

- For undirected graphs, there is no distinction between forward and back edges.
- By convention they are all called back edges.
- Furthermore, there are no cross edges (can you see why not?)

DFS-Cycles

DFS - Cycles

- The time stamps given by DFS allow us to determine a number of things about a graph or digraph.
- For example, we can determine whether the graph contains any cycles.
- We do this with the help of the following two lemmas.

DFS-Cycles

DFS - Cycles

Lemma:

- Given a digraph G = (V, E), consider any DFS forest of G and consider any edge (u, v) $\in$ E.
- If this edge is a tree, forward or cross edge, then f[u] > f[v].
- If this edge is a back edge, then f[u] $\leq$ f[v].

DFS-Cycles

DFS - Cycles

Proof:

- For tree, forward and back edges the proof follows directly from the parenthesis lemma.
- For example, for a forward edge (u, v), v is a descendent of u and so v's start-finish interval is contained within u's implying that v has an earlier finish time.

DFS-Cycles

DFS - Cycles

Proof:

- For a cross edge (u, v) we know that the two time intervals are disjoint.
- When we were processing u, v was not white (otherwise (u, v) would be a tree edge), implying that v was started before u.
- Because the intervals are disjoint, v must have also finished before u.

DFS-Cycles

DFS - Cycles

Lemma:

- Consider a digraph G = (V, E) and any DFS forest for G.
- G has a cycle if and only if the DFS forest has a back edge.

DFS-Cycles

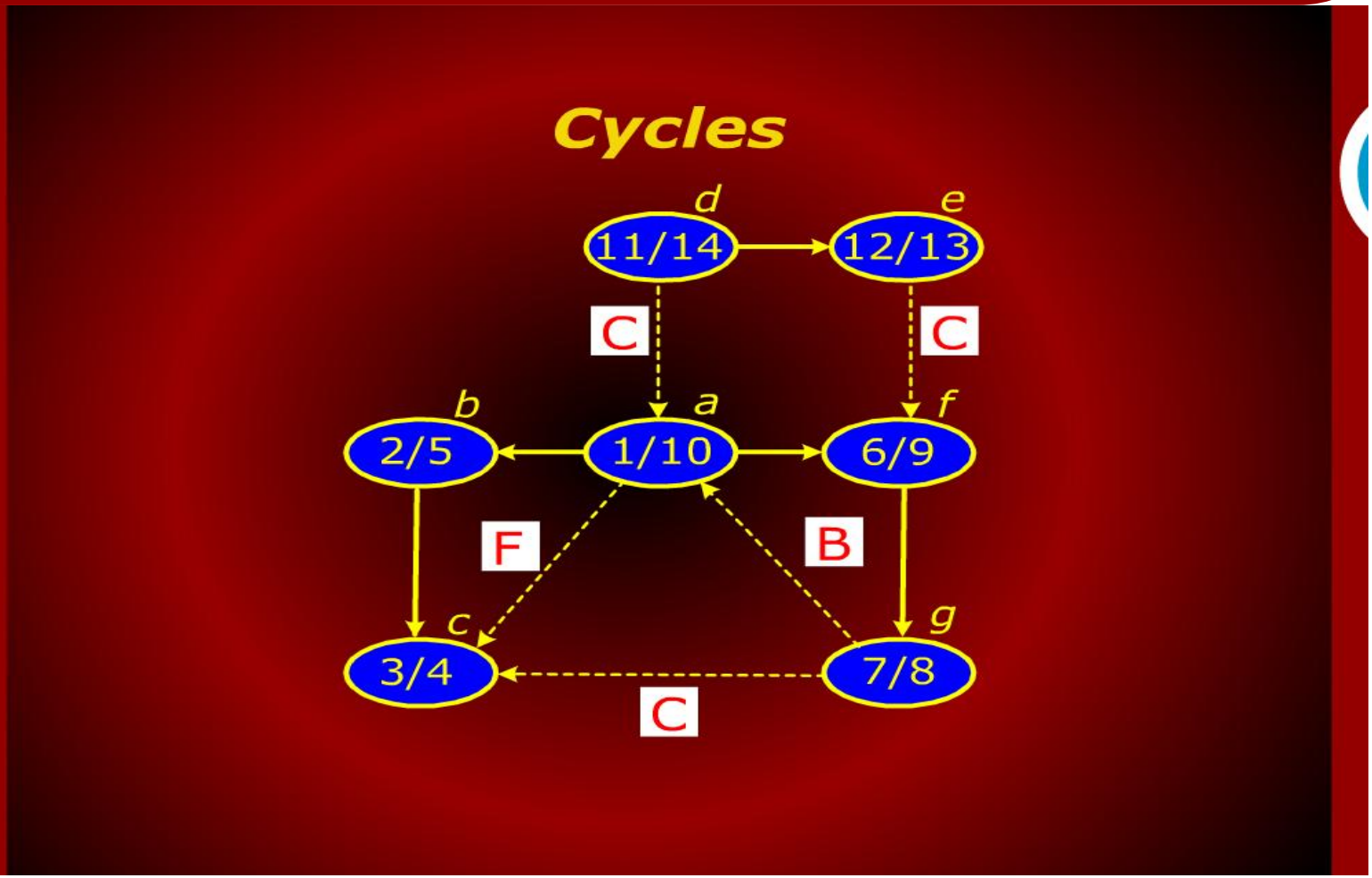
DFS - Cycles

Proof:

- If there is a back edge (u, v) then v is an ancestor of u and by following tree edge from v to u, we get a cycle.

beware: There is no relationship between number of back edges and number of cycles.

Cycles



Direct Acyclic Graph

Directed Acyclic Graph

- A directed acyclic graph, DAG for short, arise in many applications where there are precedence or ordering constraints.
- There are a series of tasks to be performed and certain tasks must precede other tasks.
- For example, in construction, you have to build the first floor before the second floor but you can do electrical work while doors and windows are being installed.

Direct Acyclic Graph

Directed Acyclic Graph

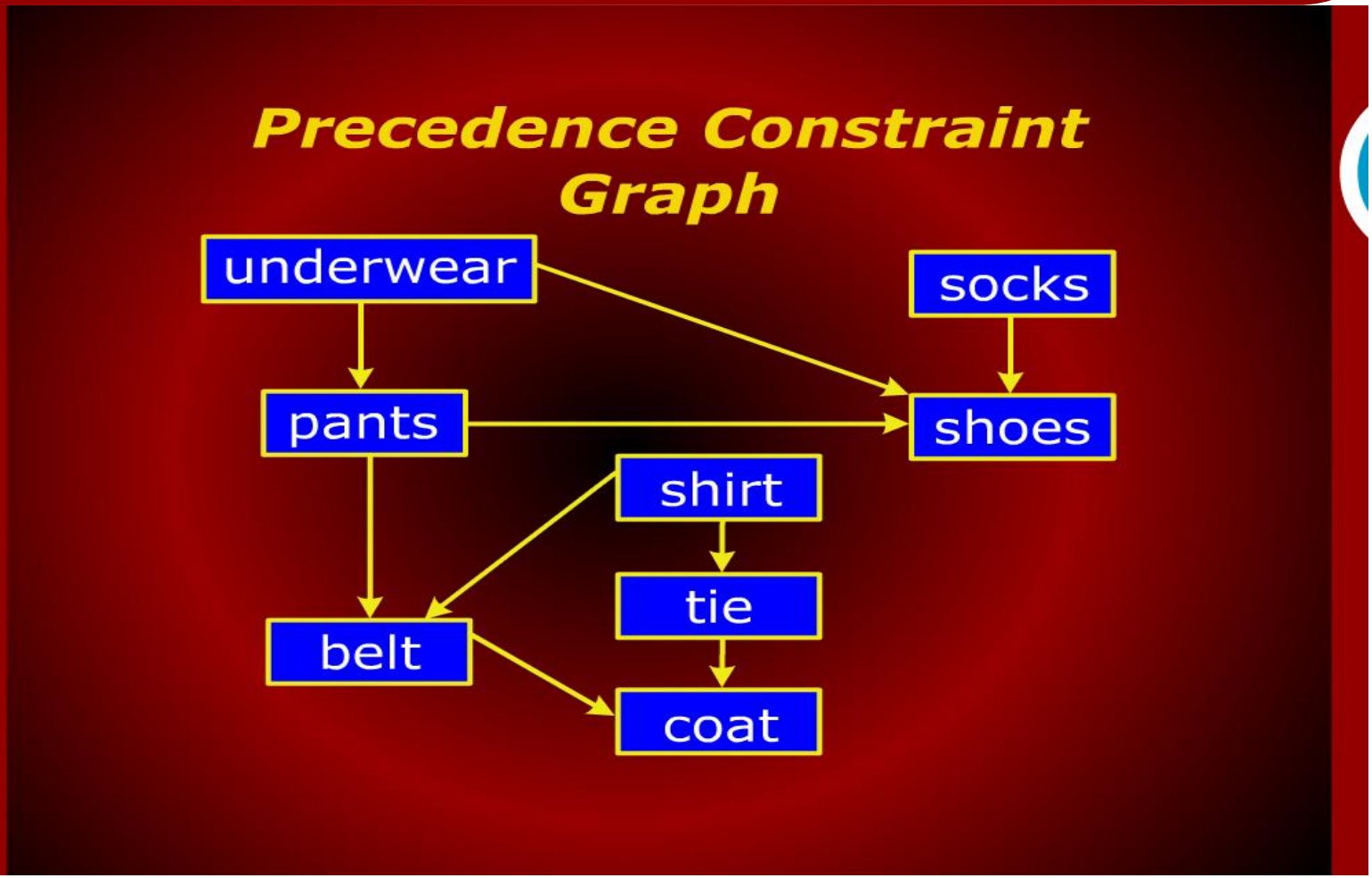
- In general, a precedence constraint graph is a DAG in which vertices are tasks and the edge (u, v) means that task u must be completed before task v begins.
- A topological sort of a DAG is a linear ordering of the vertices of the DAG such that for each edge (u, v), u appears before v in the ordering.

Topological Sort

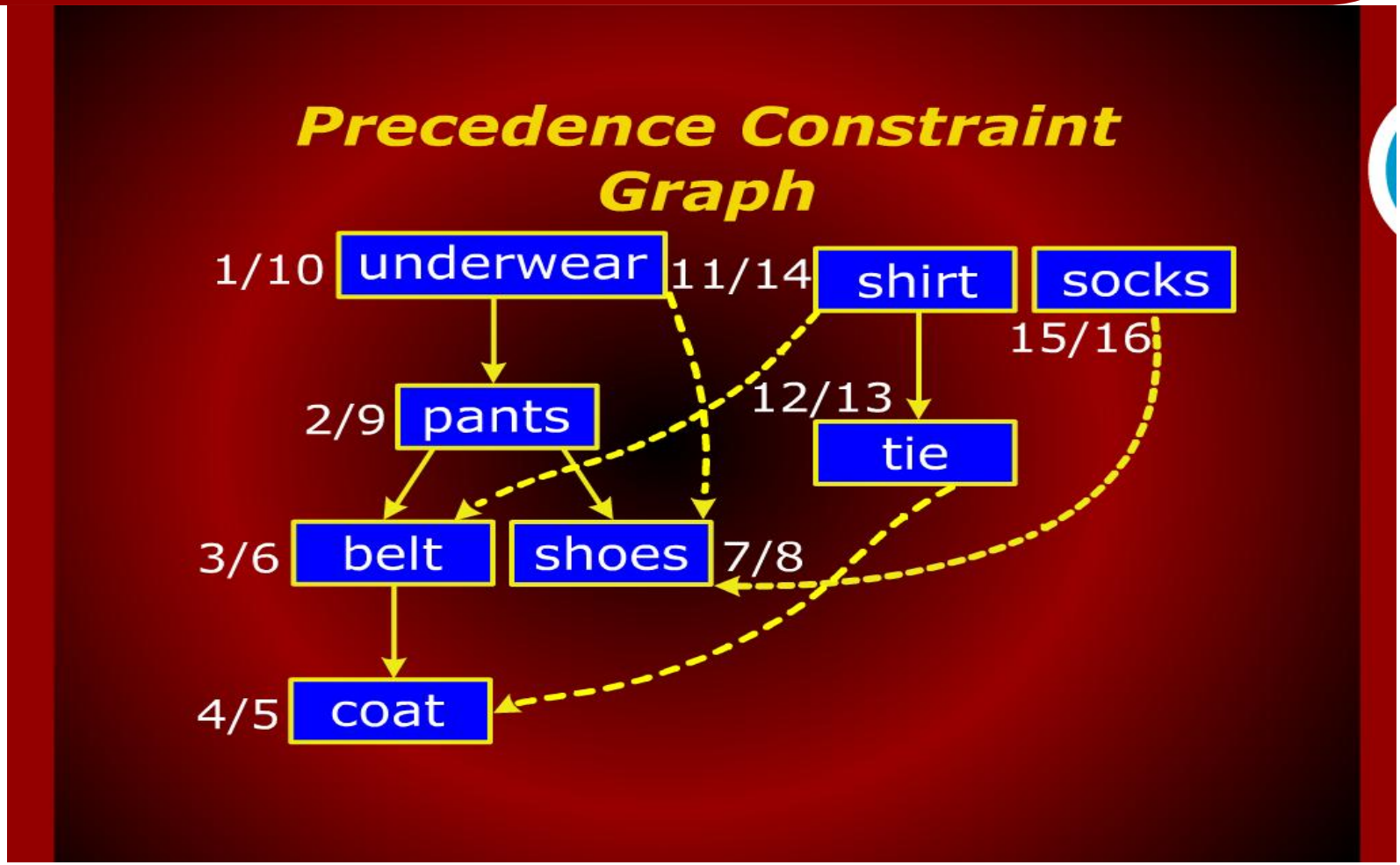
Topological Sort

- Computing a topological ordering is actually quite easy, given a DFS.
- For every edge (u, v) in a DAG, the finish time of u is greater than the finish time of v (by the lemma).
- Thus, it suffices to output the vertices in the reverse order of finish times.

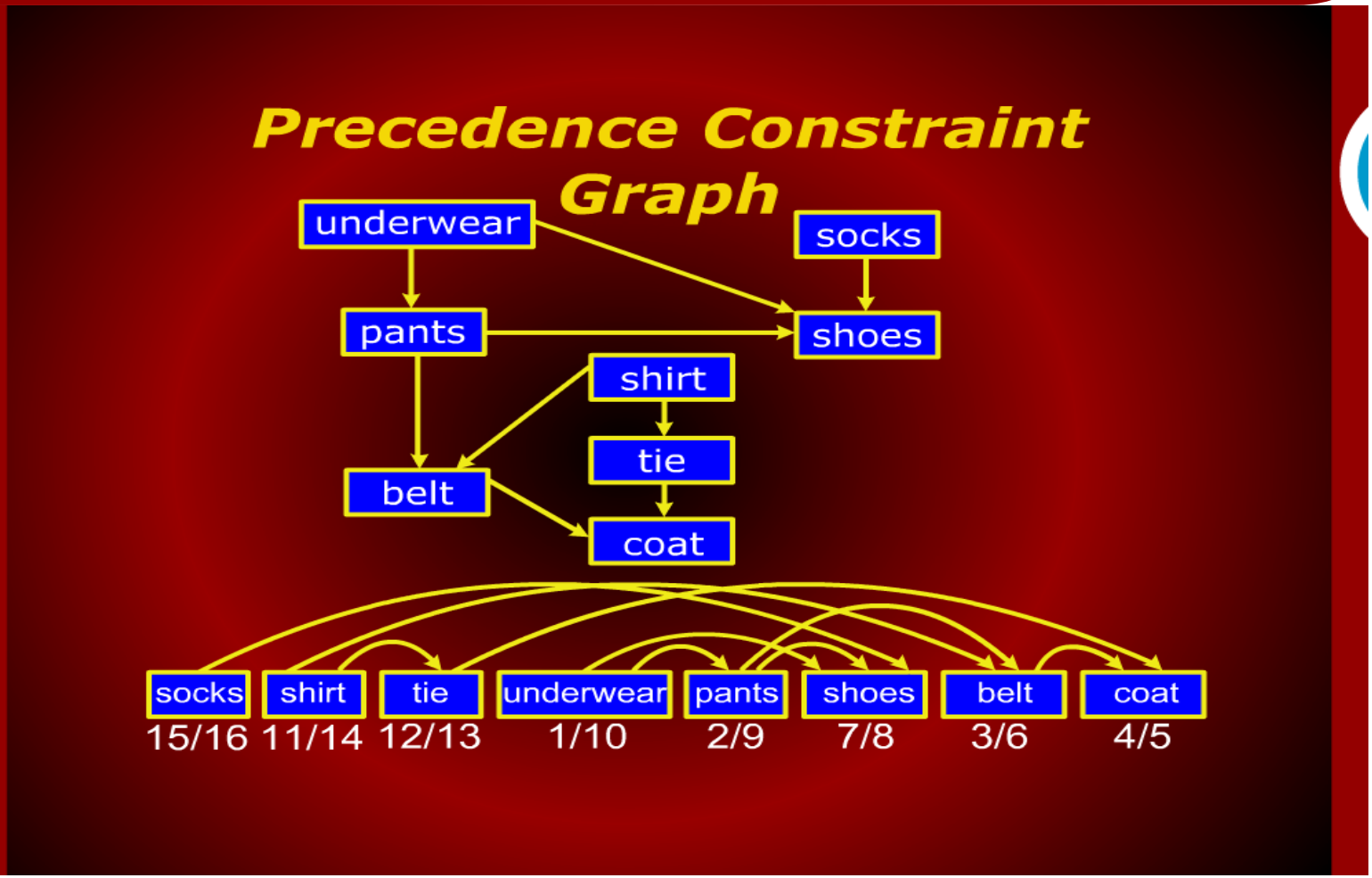
Precedence Constraint Graph



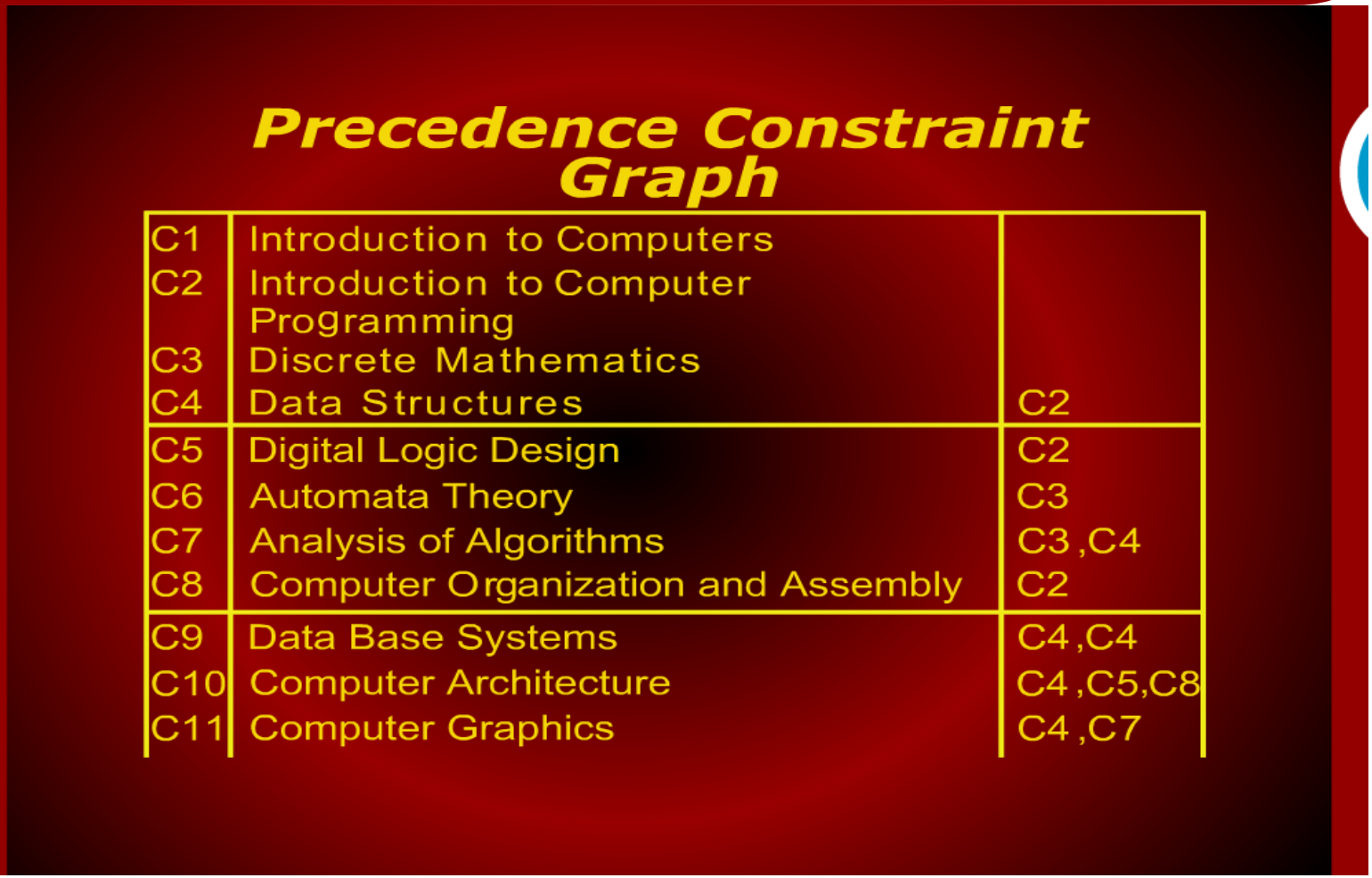
Precedence Constraint Graph



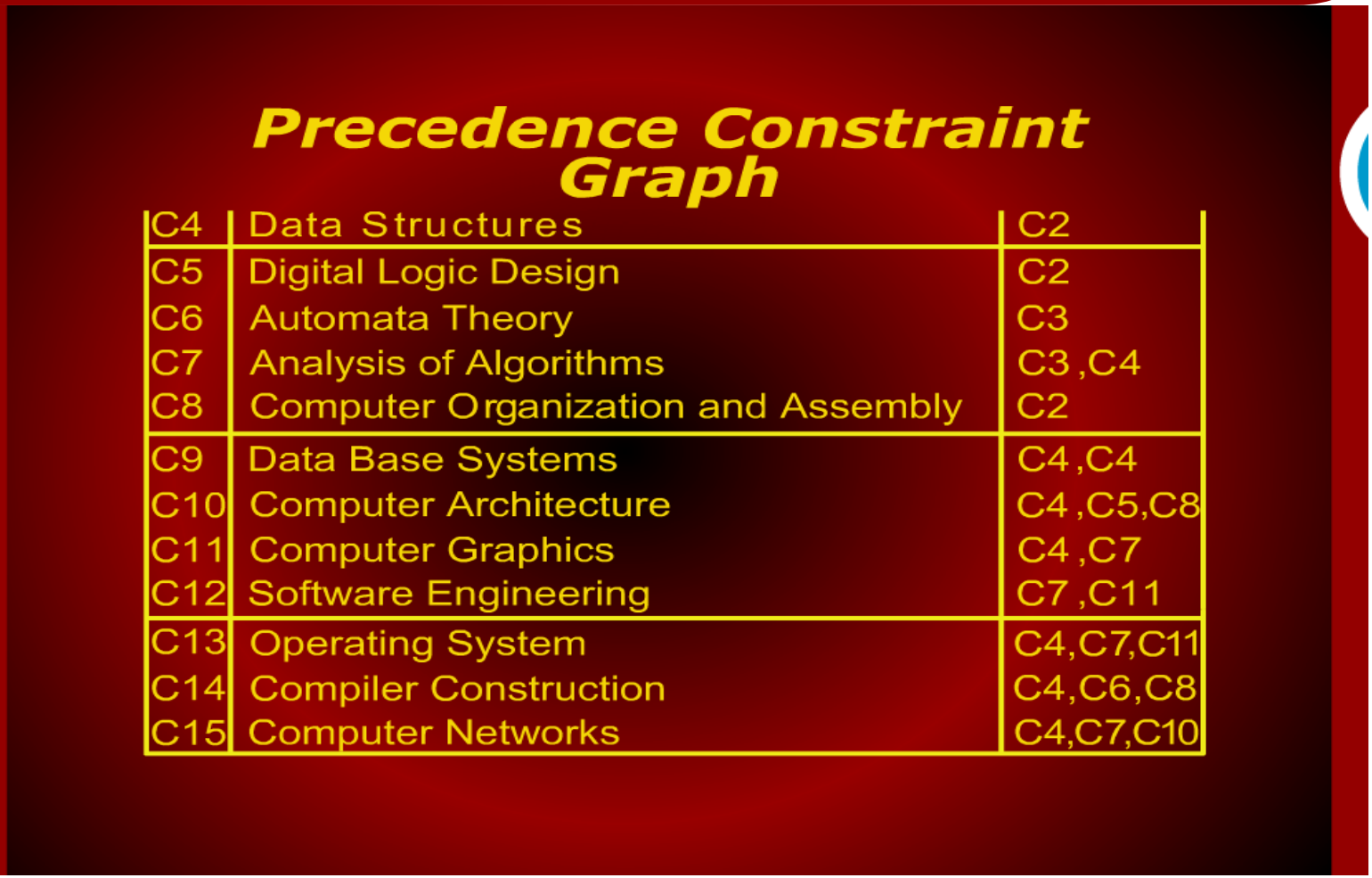
Precedence Constraint Graph



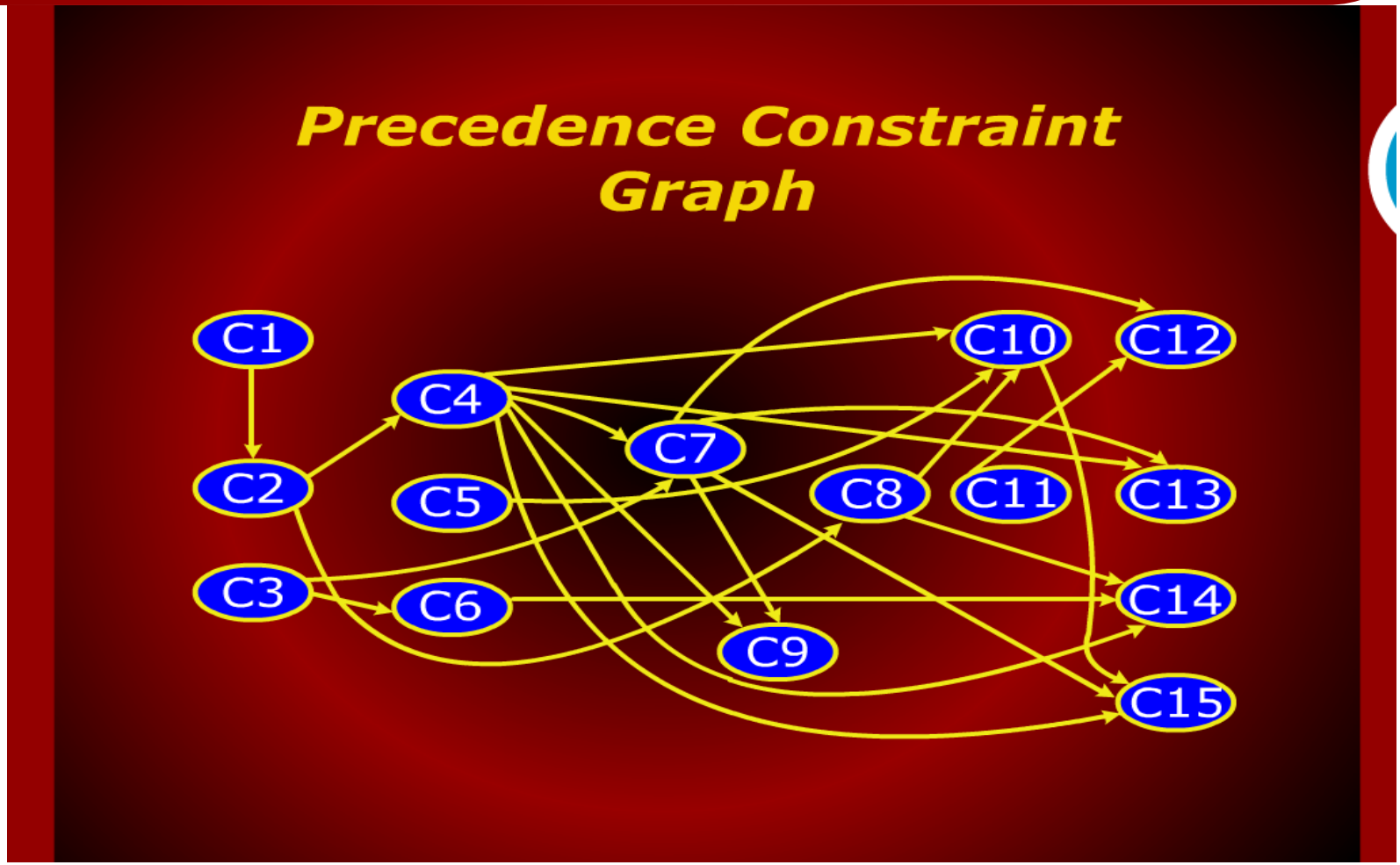
Precedence Constraint Graph



Precedence Constraint Graph



Precedence Constraint Graph

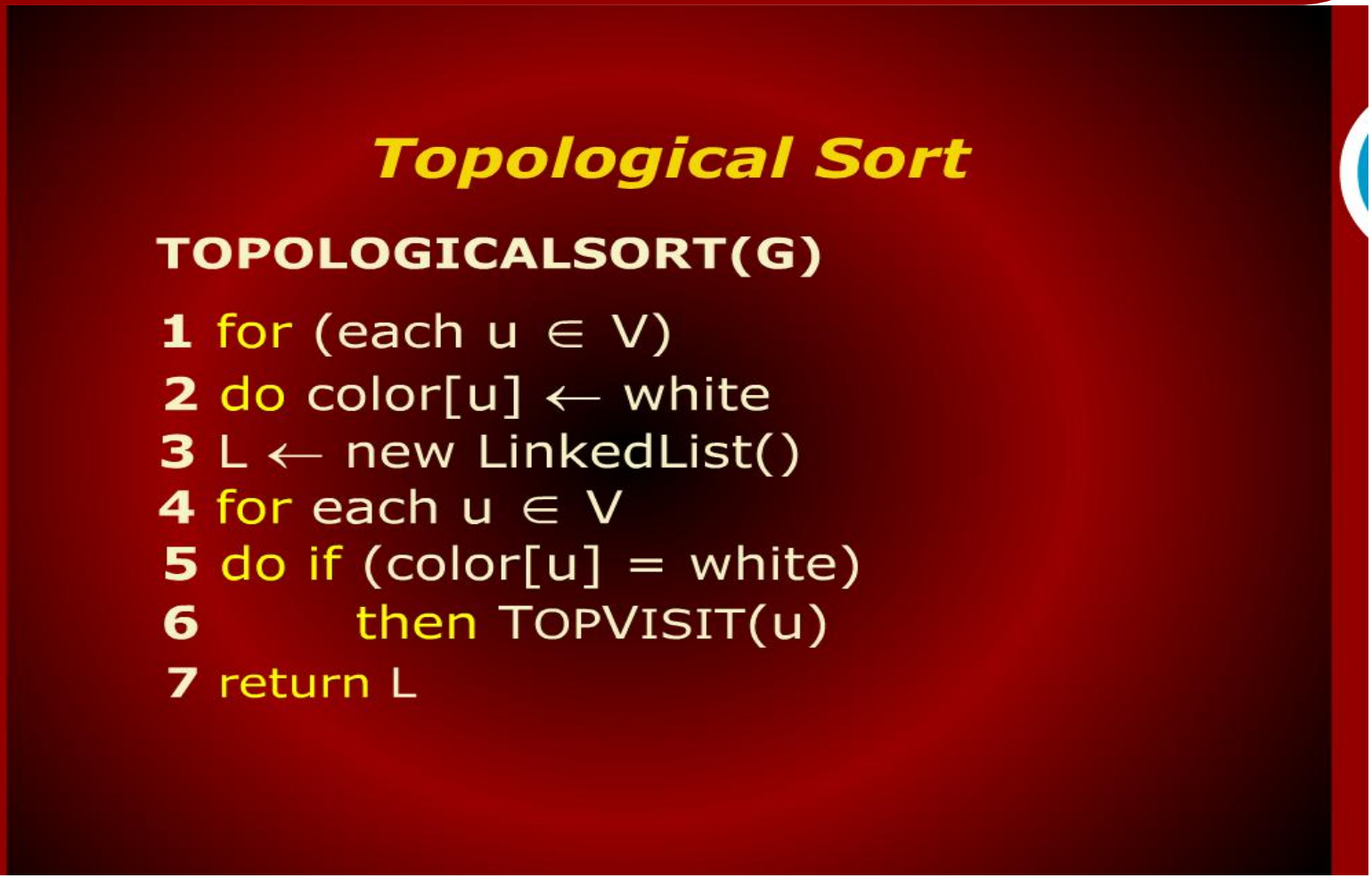


Topological Sort

Topological Sort

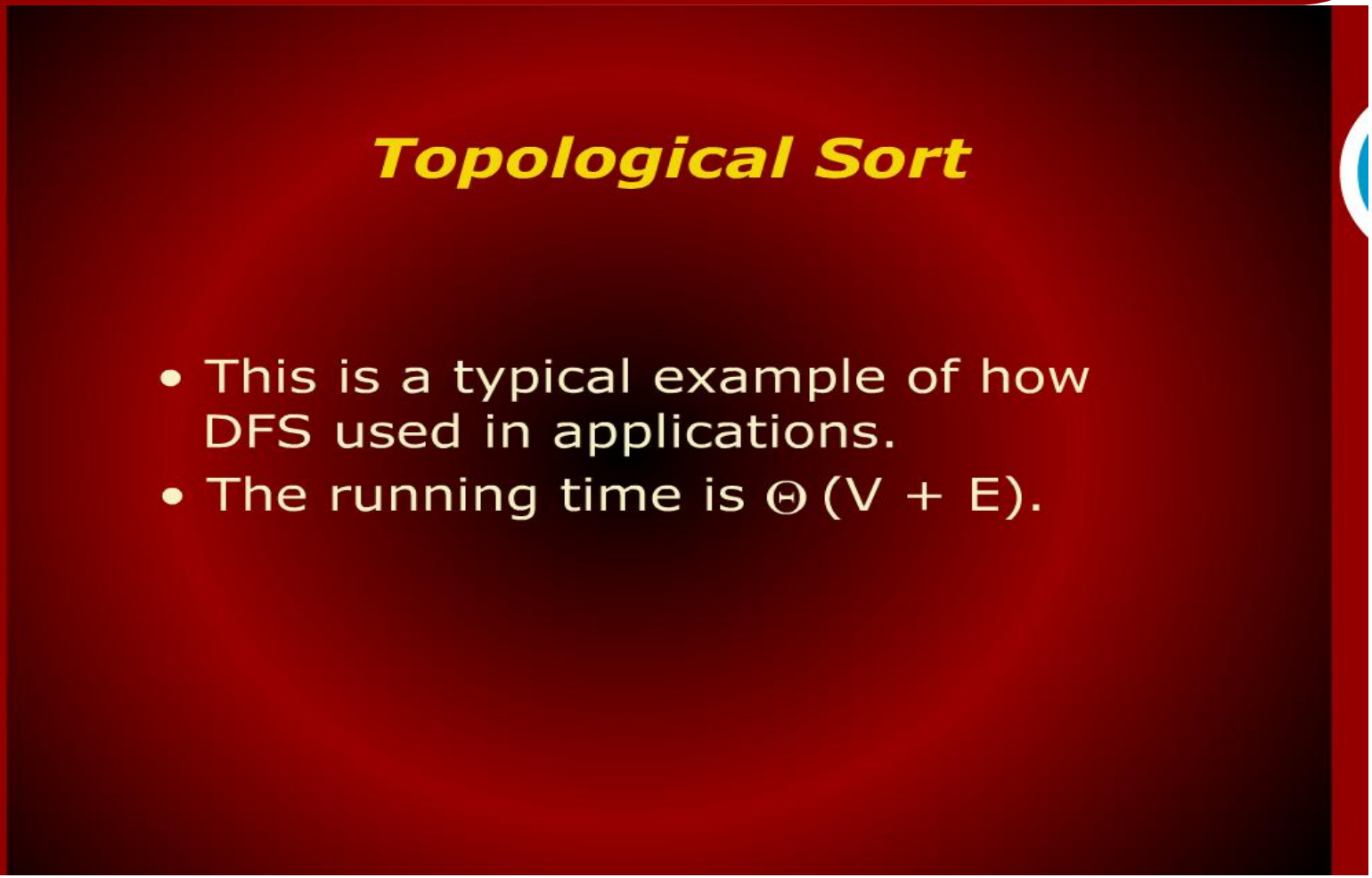
- We run DFS on the DAG and when each vertex is finished, we add it to the front of a linked.
- Note that in general, there may be many legal topological orders for a given DAG.

Topological Sort



Topological Sort

Topological Sort



Strong Components